



Máster en
Inteligencia
Artificial
UMU

Aprendizaje por Refuerzo

Sobre la entrega y defensa de la primera parte

Máster en Inteligencia Artificial
Universidad de Murcia

“El hombre que ha cometido un error y no lo corrige, comete otro error mayor.”

Confucio^a

^aReconocer y corregir los errores propios es una parte fundamental del crecimiento personal y el desarrollo moral. La idea central de esta frase tiene paralelismos interesantes con los algoritmos de aprendizaje por refuerzo.



12 de febrero de 2026

Lista de Notas Añadidas

1	NOTA: El enlace a Colab siempre funcionará.	8
2	NOTA: Rectificación sobre el nombre de los métodos.	11
3	NOTA: Los estudios forman parte de la evaluación.	11
4	NOTA: La documentación es limitada pero el contenido del Github no.	12

Índice

1	Introducción	3
2	Criterios de Evaluación	3
2.1	Informe técnico de prácticas	3
2.2	Presentación pública de trabajos	4
3	El Informe Técnico y su Defensa	5
3.1	Componente Escrita	5
3.2	Componente Experimental	6
3.3	Presentación del Informe Técnico por el Aula Virtual	7
3.4	Componente Oral	8
4	Partes del Trabajo	9
4.1	Grupo 1. Bandido de k-brazos	9
4.1.1	Plateamiento del Problema	9
4.1.2	Tareas a realizar	9
4.2	Grupo 2. Aprendizaje en entornos complejos	12
5	Anexos	15
5.1	Ejemplo reales de uso del k-bandido	15
5.2	Uso básico de Gymnasium	16
5.3	Usar PyTorch para Definir una QNet	18
5.4	Una Recomendación para Semillas Fijas	19

1. Introducción

El aprendizaje por refuerzo es una de las ramas más fascinantes de la inteligencia artificial porque está demostrando que cada vez tiene más aplicaciones, desde juegos y finanzas en sus inicios hasta la construcción LLMs (Large Language Models) como DeepSeek. Su enfoque se basa en la interacción de un agente con un entorno para aprender a tomar decisiones óptimas a través de la experiencia y la retroalimentación.

El objetivo de su trabajo final es que desarrollen las actividades prácticas que se proponen para que profundicen en algunos conceptos y fundamentos clave del aprendizaje por refuerzo y que constituyen el sistema de evaluación *SE1: Informe técnico de prácticas de laboratorio*. Dicho informe deberá ser defendido de acuerdo al sistema de evaluación *SE3: Presentación pública de trabajos*.

Más allá del desarrollo de los ejercicios que se proponen, en un máster es importante la búsqueda y selección de bibliografía relevante y el análisis de los resultados obtenidos. Con todo ello, tal y como se indica en la guía docente, se trata de que entreguen un trabajo donde muestren creatividad, precisión, rigor técnico y profundidad de análisis, discusión de resultados, justificación de las decisiones adoptadas, dando conclusiones razonadas y avaladas por fuentes acordes al informe realizado.

Por supuesto no es menos importante ser capaz de presentar el proyecto realizado y compartir las investigaciones obtenidas o trabajos académicos consultados. Durante las sesiones expositivas, los estudiantes deben ser capaces tanto de mostrar sus hallazgos como de responder a las preguntas y discusión con sus compañeros y el instructor.

A continuación se indican las actividades prácticas que debe desarrollar en grupos.

así como varias alternativas de trabajos que pueden exponer y presentar como complemento a la actividades anteriores.

2. Criterios de Evaluación

De acuerdo al sistema de evaluación de la asignatura, según la guía docente, se tienen los sistemas de evaluación:

- ▶ *SE1*: Informe técnico de prácticas de laboratorio.
- ▶ *SE3*: Presentación pública de trabajos.

2.1. Informe técnico de prácticas

El Informe técnico de prácticas de laboratorio es un documento escrito que describe el proceso, progreso y resultados de los trabajos prácticos de laboratorio, apoyado por una documentación experimental (software+notebooks). La evaluación de la documentación abarca desde el proceso de la motivación, diseño y experimentación hasta las conclusiones e incluye aspectos de comunicación y dominio disciplinar. Los criterios a destacar son:

- ▶ **Claridad y coherencia en la redacción.** Evalúa la capacidad de comunicar ideas de manera clara, ordenada y sin ambigüedades. Incluye gramática, ortografía, puntuación y estilo adecuado al contexto académico. Se observa la estructura lógica del documento (introducción, contexto, desarrollo, experimentación, conclusión, trabajos futuros y referencias).
- ▶ **Precisión técnica y dominio del contenido.** La precisión y relevancia de la información teórica presentada (estado del arte). La corrección de los procedimientos, cálculos, análisis y uso de terminología técnica. Debe evidenciarse el conocimiento específico de la asignatura.
- ▶ **Desarrollo del proceso experimental.** Con rigor metodológico describe de manera justificada, detallada y comprensible las etapas del trabajo de laboratorio. La capacidad de documentar rigurosamente lo realizado para su reproducibilidad. Calidad y pertinencia de las gráficas, análisis estadísticos, etc .. que sean claros, justos y necesarios para su discusión.
- ▶ **Capacidad de análisis, discusión y calidad de las conclusiones.** Examina la habilidad para interpretar resultados, identificar errores o desviaciones, contrastar con lo esperado. Incluye reflexión crítica, argumentación sólida y formulación de conclusiones relevantes de calidad que respondan a los objetivos iniciales de manera concisa. Considera sugerencias para trabajos futuros.

- ▶ **Uso de referencias cruzadas, fuentes y referencias bibliográficas.** Realiza las referencias adecuadas (cruzadas o bibliográficas) en cada parte del documento. Evalúa la calidad, actualidad y relevancia de las fuentes utilizadas. Considera el cumplimiento de la norma de citación que se pida para la entrega (p.e. APA, IEEE, etc.).

! Importante

- ▶ La presentación y defensa del informe no obliga a presentar todas las partes.
- ▶ La puntuación máxima que se puede obtener en cada parte es de 3.5 puntos.

2.2. Presentación pública de trabajos

Se trata de exponer y evaluar los resultados del trabajo realizado y suele venir acompañado de una discusión final y debate con el público, moderado por el docente. Permite evaluar competencias de expresión oral y comunicación, adicionales a las específicas de la asignatura.

La evaluación de la exposición oral estará basada en el informe técnico correspondiente y busca evaluar tanto las competencias comunicativas orales como el dominio del contenido técnico que se presentará. Criterios a destacar son:

1. **Claridad y estructura de la presentación oral.** La exposición tiene una introducción clara, un desarrollo ordenado y una conclusión adecuada. El mensaje debe ser comprensible, independientemente del estilo de habla. Se prioriza al claridad conceptual y metodológica. Los tiempos entre los miembros del grupo están bien distribuidos.
2. **Dominio del contenido y rigor técnico.** Cada miembro expone contenidos de dificultad similar y prioriza la información relevante. Muestra comprensión de los conceptos presentados. Se explica, sin ambigüedades, el proceso experimental, resultados y conclusiones sin depender excesivamente de notas. Precisión en las explicaciones y el uso adecuado de terminología técnica.
3. **Cobertura integral del informe técnico.** La exposición oral **refleja** de manera sintética pero completa **el contenido del informe técnico final**. Se valorará positivamente la capacidad de comunicar todas las secciones clave del trabajo escrito a presentar. Debe evitarse la omisión de apartados relevantes respecto del informe final.

Nota: Aunque para la exposición no es necesario mostrar todo lo que se vaya a presentar en el informe, cuanto más se asemeje la exposición al documento final, mayor valoración tendrá este criterio.

4. **Uso de apoyos visuales o materiales complementarios.** Calidad y pertinencia de las diapositivas, gráficos, tablas, animaciones, vídeos o materiales visuales. Adecuada integración de los elementos visuales en la exposición. Uso responsable de tecnología (p.e. no leer directamente de las diapositivas, evitar saturación de texto, etc).
5. **Gestión de preguntas, discusión y argumentación.** **Todos** los miembros del grupo son capaces de responder con claridad, seguridad y precisión a preguntas del público o del docente. Demuestran pensamiento crítico y dominio del tema. Actitud receptiva y argumentación basada en evidencia del trabajo realizado.

! Importante

- ▶ Se deberá presentar **la misma versión** del repositorio con la que hayan realizado la parte documental.
- ▶ Solo cuando se haga la entrega y se realice la correspondiente defensa de la entrega, el alumno obtendrá una calificación de dicha parte.
- ▶ Presentar el trabajo pero no defenderlo, o defender un trabajo que no se presenta, no conllevará ninguna calificación.
Por favor, evite esta situación para no perjudicar la exposición de sus compañeros. Si no se presenta, sus compañeros podrán tener más tiempo para defender mejor sus trabajos.

3. El Informe Técnico y su Defensa

El informe técnico es un documento escrito que describe el proceso, progreso y resultados de los trabajos prácticos de laboratorio. También deberá incluir discusiones y conclusiones del trabajo, así como estado del arte y referencias bibliográficas. Es fundamental que consulten fuentes confiables, como artículos científicos, libros y recursos académicos, y que los integren adecuadamente en sus análisis. Recuerde lo siguiente: *Todos los recursos y materiales no originales que se utilicen en los ejercicios evaluables, incluyendo herramientas de Inteligencia Artificial, ayuda de compañeros, recursos de internet, libros, artículos, etc. deberán referenciarse claramente en el código fuente y la documentación asociada a los ejercicios prácticos entregados. De otro modo será de aplicación el Artículo 22.1 del REVA.*

El trabajo constará de **3** partes:

- ▶ Parte 1: Problema del Bandido de k-brazos.
- ▶ Parte 2: Técnicas tabulares de Aprendizaje por Refuerzo.
- ▶ Parte 3: Técnicas aproximadas de Aprendizaje por Refuerzo.

El informe tendrá dos fuentes documentales:

- ▶ Un documento escrito, que será un fichero .pdf que expondrá todo el proyecto realizado junto con el análisis de la componente experimental.
- ▶ Toda la documentación experimental (notebooks), que se ejecutará en Colab (**sin errores**) a partir de un repositorio de GitHub y que respalde al documento escrito.

Además, se deberá de hacer una defensa pública del trabajo realizado.

! Importante

- ▶ Para tener una evidencia completa, a efectos de evaluación y custodia, en el Aula Virtual tendrán que presentar todas las partes con todas sus componentes en **un único fichero .zip**. Tendrá como fecha tope de entrega el **15 de marzo**, y se presentará por el Aula Virtual (herramienta Tareas). Tenga en cuenta en cuenta que:
 - Se deberá presentar en el fichero .zip **la misma versión** del repositorio con la que hayan realizado la parte documental a efectos de poder reproducir los resultados.
 - La entrega del código no le exime de la exigencia de que se pueda ejecutar en Colab.
- ▶ Las exposiciones del informe técnico, que serán grabadas, tendrán lugar el **2 y el 5 de marzo**. Las horas de esos días se dedicarán en exclusividad a las exposiciones, sin importar el número de presentados.

3.1. Componente Escrita

Es importante que parte escrita (el fichero .pdf) se estructure de manera clara. Para ello, en la portada se pondrá un **título** y a **los autores**. Incluya el nombre de la obra y los nombres completos de los autores del trabajo, junto con el correo electrónico um.es para cada autor.

Además, para cada parte deberá incluir, en términos generales, los siguientes apartados:

1. Introducción:

- ▶ Breve descripción del problemas y su relevancia.
- ▶ Motivación del trabajo: ¿Por qué es importante el problema estudiado?
- ▶ Objetivos del informe: ¿Qué se pretende lograr con el desarrollo del trabajo?
- ▶ Organización del documento: breve explicación de la estructura del informe.

2. Desarrollo:

- ▶ Definición formal del problema o aplicación específica abordada.
- ▶ Describir enfoques existentes en la literatura sobre problemas similares.
- ▶ Contexto y antecedentes necesarios para entender el trabajo.
- ▶ Explicación de los métodos utilizados para abordar el problema.
- ▶ Justificar cómo el presente trabajo se diferencia o mejora enfoques previos.

3. Algoritmos:

- ▶ Indicar los pasos de los algoritmos empleados.
 - ▶ Expresarlos en pseudocódigo y, opcionalmente, diagramas de flujo si es necesario.
 - ▶ Justificar la elección de los algoritmos y su aplicabilidad al problema estudiado.
4. **Evaluación/Experimentos:**
- ▶ Configuración experimental: herramientas, entornos de prueba, datasets empleados, etc.
 - ▶ Métricas utilizadas para evaluar el desempeño del método propuesto.
 - ▶ Resultados obtenidos, presentados en tablas o gráficos según corresponda.
 - ▶ Análisis crítico de los resultados y comparación con otros enfoques.
5. **Conclusiones:**
- ▶ Un resumen de los principales resultados obtenidos.
 - ▶ Limitaciones del estudio y posibles mejoras futuras.
 - ▶ Reflexión sobre la importancia del trabajo y su impacto en el campo del aprendizaje por refuerzo.
 - ▶ Líneas *futuras* de estudio.
6. **Bibliografía:** Listar todas las referencias utilizadas en el trabajo. Se recomienda utilizar un gestor de referencias como BibTeX para facilitar la organización de las citas. Se recomienda en las referencias el enlace URL donde se encuentre el repositorio GitHub donde reproducir lo documentado.

! Importante

- ▶ Desarrolle los apartados teniendo en cuenta los criterios de evaluación.
- ▶ El documento se desarrollará, necesariamente, **de acuerdo a la plantilla** que tienen en recursos del Aula Virtual.
- ▶ Sean siempre **claros y concisos** en su informe, enfocándose en los **puntos clave** sin omitir de mencionar y explicar los **detalles importantes**.
- ▶ Cada apartado podrá contener subapartados; pero cada (sub)apartado del documento se desarrollará con la extensión que sea acorde a lo que se presenta. No desarrolle (sub)apartados con contenidos innecesarios.
- ▶ Si lo prefiere, puede crear un documento con solo dos partes: (1) el bandido de k-brazos, y (2) Aprendizaje en entornos complejos y en este último distinguir entre métodos tabulares y aproximados.
- ▶ La extensión del documento escrito (.pdf) no superará las ~~10 páginas~~ **10 folios**, descontada la carátula, índice, la bibliografía y anexos (si procedieran).

3.2. Componente Experimental

Cada parte tendrá su componente experimental que respaldará la componente documental explicada en el apartado anterior. Cada componente experimental constará de tantos notebooks y ficheros .py como considere necesarios, siempre que sirvan de respaldo a lo documentado.

En definitiva, la componente escrita (fichero .pdf) tendrá su correspondiente respaldo en **GitHub** y su contenido debe seguir las siguientes instrucciones:

1. Crear un repositorio llamado **NombreGrupo**, donde **NombreGrupo** es el nombre del grupo. Por ejemplo, los primeros apellidos de sus miembros.
2. Incluir los *notebooks* **ya resueltos** (ejecutados) en el repositorio.
3. Incluir **en el repositorio** un fichero README.md con los siguientes datos.

```
# Título del Trabajo

## Información
- **Alumnos:** Apellido1, Nombre1; Apellido2, Nombre2; Apellido3, Nombre3
- **Asignatura:** Nombre de la asignatura
- **Curso:** 2025/2026
- **Grupo:** NombreGrupo (p.e. ap1ap2ap3m, usando los apellidos)

## Descripción
```

[Breve descripción del trabajo y sus objetivos]

Estructura

[Explicación de la organización del repositorio]

Instalación y Uso

[Instrucciones si son necesarias]

Tecnologías Utilizadas

[Lista de lenguajes, frameworks, etc.]

4. El repositorio tendrá la siguiente estructura general para cada parte:

```
trabajo/
|__ src/      # Código .py (expanda src, o considere varios src con distintos nombres)
|__ docs/     # Documentación ( .bib, .pdf, ... si procede)
|__ tests/    # Tests (si aplica)
|__ data/     # Datos necesarios (si aplica)
|__ README.md
|__ main.ipynb # Fichero principal
|             # Breve introducción del problema y enlace a los estudios.
|__ notebook1.ipynb # P.e. presentación de métodos tabulares
|__ notebook2.ipynb # P.e. presentación de métodos aproximados
|__ etc.ipynb  # Recuerde poner tantos como estudios realizados
```

Utilice **nombre significativos** en cada uno de los ficheros y añada cuantos ficheros README.md sean necesarios para aclarar el contenido de cada directorio.

5. Todo se podrá ejecutar en Colab.

El fichero **main.ipynb** comenzará con el típico enlace de  [Open in Colab](#), que abrirá el notebook en Colab. https://colab.research.google.com/github/autores/trabajo_NombreGrupo/blob/main/main.ipynb Las primeras celdas del *notebook* principal harán una copia del repositorio y se instalarán los paquetes que sean necesarios. A partir de dicho *notebook* se podrá navegar entre los distintos *notebooks*.

! Importante

Tenga en cuenta los siguiente:

- ▶ El enlace  [Open in Colab](#) se consigue de forma automática al salvar el **notebook** desde Colab en el repositorio.
- ▶ Intente usar la misma estructura que en el documento .pdf. Por ejemplo, si distingue 3 partes en el documento .pdf, distinga 3 partes en el directorio **src** y al menos 3 **notebooks** con nombre identificativos de cada parte.
- ▶ Asegúrese de que cada **notebook** tenga la **semilla** de generación de números aleatorios que permita reproducir las gráficas o tablas de las componentes documentadas.
- ▶ El **prerequisito** para la evaluación es que al entrar por primera vez en el *notebook* y seleccionar la opción **Entorno de ejecución > Ejecutar todas** **no se produzcan errores** y el resultado final sea exactamente el mismo al que se encuentra en el repositorio. Es decir, que sus notebooks y lo documentado **son reproducibles**.
 - Tendrán que garantizar que las celdas iniciales de cada *notebook* realicen todo lo necesario para que no se produzca error alguno en su ejecución.
 - La no-ejecución completa de un notebook, conlleva que todo su contenido quedará descartado y no se considerará superado.

3.3. Presentación del Informe Técnico por el Aula Virtual

En el Aula Virtual deberá de presentar la evidencia del trabajo realizado en **un único fichero .zip** que contenga el fichero .pdf de la componente escrita y el repositorio completo que respalda al documento.

De acuerdo a lo explicado anteriormente, se espera que en la descompresión del fichero .zip se muestre una estructura general similar a la Figura 1.

```
NombreGrupo (github)
|-- informe.pdf      # Informe técnico (se espera no más de 10 folios de contenido)
|-- main.ipynb       # Fichero principal que accede a los distintos main.ipynb de cada parte
|-- k_brazos
|   |-- src/         # Código .py (expanda src, o considere varios src con distintos nombres)
|   |-- docs/        # Documentación ( .bib, .pdf, ... si procede)
|   |-- tests/       # Tests (si aplica)
|   |-- data/        # Datos necesarios (si aplica)
|   |-- README.md
|   |-- main.ipynb   # Fichero principal de esta parte
|   |--             # Breve introducción del problema y enlace a los estudios.
|   |-- notebook1.ipynb # P.e. presentación de métodos epsilon-greedy
|   |-- etc.ipynb    # Recuerde poner tantos como estudios realizados
|-- Entornos_Complejos
|   |-- src/         # Código .py (expanda src, o considere varios src con distintos nombres)
|   |-- docs/        # Documentación ( .bib, .pdf, ... si procede)
|   |-- tests/       # Tests (si aplica)
|   |-- data/        # Datos necesarios (si aplica)
|   |-- README.md
|   |-- main.ipynb   # Fichero principal de esta parte
|   |--             # Breve introducción del problema y enlace a los estudios.
|   |-- notebook1.ipynb # P.e. presentación de métodos tabulares
|   |-- etc.ipynb    # Recuerde poner tantos como estudios realizados
```

Figura 1: Estructura esperada del fichero .zip si considera que su documento .pdf se divide en dos partes

! Importante

Es fundamental que todo el fichero .zip se descomprima **a partir de un directorio** cuyo nombre se identifique con el nombre del grupo.

NOTA

!! El enlace a Colab siempre funcionará. *No importa si el repositorio lo tenéis público o privado. Eso no afectará a la comprobación del código. Lo importante, y **se insiste** en ello, es que esté el enlace a Colab y se respete en el fichero .zip la misma estructura (y ficheros actualizados) que se tiene en el repositorio con el que se han realizado las pruebas.*

3.4. Componente Oral

En los apartados anteriores se ha mostrado en qué consiste la componente escrita, la componente experimental y cómo se deben de presentar ambas en el Aula Virtual. Pero recuerde que debe de hacer una exposición pública de lo presentado.

En este sentido, de acuerdo al número de alumnos matriculados, si todos se presentaran a la exposición por videoconferencia (zoom), cada uno tendrá una intervención de unos 4', supuesto que no hay descansos y no hay problemas técnicos.

Para la exposición puede utilizar cuantos recursos sean necesarios siempre que sean visibles a través de la aplicación Zoom (transparencias, vídeos, animaciones, etc). Tenga en cuenta los criterios de evaluación que se han indicado en la sección 2.2.

! Importante

A efectos de conocer el tiempo real de exposición que tendrá cada grupo, deberán notificar, a más tardar el **28 de febrero** y antes de las 14h, quiénes presentarán el trabajo completo. El mismo día 28 se anunciará el orden de defensa de los trabajos, que se seleccionarán de forma aleatoria.

Recuerde que las exposiciones del informe técnico, que serán grabadas, tendrán lugar el **2 y el 5 de marzo**.

4. Partes del Trabajo

Como se ha indicado anteriormente, el trabajo constará de **3** partes:

- ▶ Parte 1: Problema del Bandido de k -brazos.
- ▶ Parte 2: Técnicas tabulares de Aprendizaje por Refuerzo.
- ▶ Parte 3: Técnicas aproximadas de Aprendizaje por Refuerzo.

A efectos conceptuales se describe cada una de ellas dividiendo las técnicas en dos grupos:

- ▶ **Grupo 1.** Aprendizaje en el bandido de k -brazos
- ▶ **Grupo 2.** Aprendizaje en entornos complejos

4.1. Grupo 1. Bandido de k -brazos

El problema del bandido multibrazo es el mejor ejemplo del dilema de la **Exploración-Explotación** en el contexto del aprendizaje por refuerzo. Los mejores algoritmos son los que son capaces de encontrar el mejor equilibrio entre ambas opciones.

4.1.1. Planteamiento del Problema

El **problema del bandido de k -brazos** modela una situación de toma de decisiones secuencial bajo incertidumbre. Consideraremos entorno estacionarios. Consiste en lo siguiente:

- ▶ Una máquina tragaperras con k brazos (acciones)
- ▶ Cada brazo i tiene una distribución de recompensa desconocida $\mathcal{P}_i$
- ▶ En cada paso t , el agente:
 1. Elige un brazo $a_t \in \{1, \dots, k\}$
 2. Recibe una recompensa $r_t \sim \mathcal{P}_{a_t}$
- ▶ El objetivo es maximizar la **recompensa acumulada** $\sum_{t=1}^T r_t$ donde T es el horizonte temporal.

Existen varios criterios para seleccionar una alternativa. Por ejemplo:

- ▶ Una política es tomar decisiones que tengan en cuenta aquellos brazos que, por la experiencia, han mostrado tener mayores recompensas.
- ▶ Alternativamente, otra forma de medir las mejores decisiones es considerar como medida de desempeño el **rechazo** (*regret*) que cuantifica la pérdida por no elegir siempre el mejor brazo:

$$R(T) = T\mu^* - \mathbb{E} \left[\sum_{t=1}^T \mu_{a_t} \right]$$

donde $\mu^* = \max_i \mu_i$ y $\mu_i = \mathbb{E}[\mathcal{P}_i]$.

4.1.2. Tareas a realizar

En este grupo se analizan los algoritmos que resuelven el problema del bandido de k -brazos, comparando sus rendimientos. IDEALMENTE, se trataría de implementar varias familias de métodos (ϵ -greedy, UCB, ascenso del gradiente, bayesianos, etc) evaluando las recompensas y rechazos acumulados para detectar *el mejor* algoritmo dentro de cada familia.

Las tareas a realizar en esta primera parte son las siguientes.

- ▶ Construir varios tipos de brazos.
- ▶ Completar el estudio de la familia ϵ -greedy que ya ha presentado el profesor
- ▶ Implementación de otros algoritmos.
- ▶ Realizar un estudio comparativo

Tipos de bandidos

Cuando se selecciona un acción (un bandido), se retorna una recompensa que se rige por una distribución de probabilidad desconocida: $r \sim \mathcal{P}_a$

Un tipo de distribución es la **distribución Binomial**, $\mathcal{B}(n, p)$. Es cuando nos referimos a que, si reali-

zamos n acciones, el **número total de éxitos** (recompensas positivas) se dará con probabilidad p .

La distribución Bernoulli es un caso particular de la binomial, para $n = 1$. Se corresponde a considerar situaciones de éxito o fracaso como el caso del suministro de medicamentos.

Si $X \sim \mathcal{B}(n, p)$ con $np \geq 5$ y $n(1-p) \geq 5$ se garantiza que la distribución binomial no está demasiado sesgada y tenga una forma aproximadamente simétrica. Entonces se puede hacer una **aproximación normal**: $X \approx \mathcal{N}(\mu, \sigma^2)$ donde: $\mu = np$ y $\sigma^2 = np(1-p)$.

No obstante, hay problemas donde surge de forma natural el uso de una distribución Distribución Normal, $\mathcal{N}(\mu, \sigma^2)$, en cada uno de los brazos.

En la sección 5.1 tiene ejemplos de uso de estos 3 tipos de distribuciones en el contexto del bandido de k-brazos.

¿Qué debe de hacer?

Implemente los bandidos con estas distribuciones de probabilidad. Recuerde que los basados en la distribución normal ya están implementados y que la Benoulli es un caso particular de la binomial.

Completar el estudio de la familia ϵ -greedy

El notebook [bandit_experiment.ipynb](#) situado en el repositorio [ldaniel-hm/eml_k_bandit](#) localizado en [GitHub](#) es un ejemplo de cómo se pueden estudiar los algoritmos del bandido de k-brazos. En este documento se comparan tres algoritmos ϵ -greedy. Pero observa que el código del repositorio está incompleto. En esta parte queremos hacer un estudio más completo sobre este conjunto de técnicas.

Para completar el estudio vamos a **añadir más gráficos**. Se consideran las siguientes gráficas:

- (Necesaria) Mostrar el porcentaje en el que fue seleccionado el brazo óptimo.

```
def plot_optimal_selections(steps: int,
                           optimal_selections: np.ndarray,
                           algorithms: List[Algorithm]):
    """
    Genera la gráfica de Porcentaje de Selección del Brazo Óptimo vs Pasos de Tiempo.

    :param steps: Número de pasos de tiempo.
    :param optimal_selections: Matriz de porcentaje de selecciones óptimas.
    :param algorithms: Lista de instancias de algoritmos comparados.
    """
```

- (Necesaria) Mostrar la evolución del rechazo, de forma análoga a cómo se muestra la evolución de las recompensas.

```
def plot_regret(steps: int,
                regret_accumulated: np.ndarray,
                algorithms: List[Algorithm], *args):
    """
    Genera la gráfica de Regret Acumulado vs Pasos de Tiempo

    :param steps: Número de pasos de tiempo.
    :param regret_accumulated: Matriz de regret acumulado (algoritmos x pasos).
    :param algorithms: Lista de instancias de algoritmos comparados.
    :param args: Opcional. Parámetros que consideres. P.e. la cota teórica  $Cte * \ln(T)$ .
    """
```

- (Opcional) Mostrar las estadísticas de cada brazo. Cada valor en el eje X representará un brazo. En el eje Y se representa el promedio de las ganancias de cada brazo. El gráfico mostrará un histograma para cada brazo donde se muestre el promedio de las ganancias obtenidas pero en el eje X, como etiqueta, se mostrará también el número de veces que fue seleccionado el brazo e indicar si es el brazo óptimo o no.

Añade lo que considere necesario que ayude a clarificar el significado de la gráfica.

```
def plot_arm_statistics(arm_stats: LoQueConsideres,
                       algorithms: List[Algorithm], *args):
    """
    Genera gráficas separadas de Selección de Arms:
    Ganancias vs Pérdidas para cada algoritmo.
    """
```

```
:param arm_stats: Lista (de diccionarios) con estadísticas de cada brazo por algoritmo.
:param algorithms: Lista de instancias de algoritmos comparados.
:param args: Opcional. Parámetros que consideres
"""
```

NOTA

!! Rectificación sobre el nombre de los métodos. Aunque se mantiene el propósito necesario/obligatorio de cada apartado gráfico, se rectifica la definición de los métodos `plot_regret` y `plot_arm_statistics` en los dos últimos apartados. En versiones anteriores a este documento estaban intercambiados. Tal y como está ahora es lo correcto.

¿Qué debe de hacer?

- ▶ Modifique los ficheros `bandit_experiment.ipynb` y https://github.com/ldaniel-hm/eml_k_bandit/blob/main/plotting/plotting.py para incluir en el estudio original las gráficas indicadas.
- ▶ Cuando complete el estudio de la familia de algoritmos ϵ -greedy, indique cuál de todas las gráficas que se piden son las más relevantes ¿por qué es esa gráfica o esas gráficas las más importantes?

! Importante

- ▶ Realice representaciones con pocas gráficas. P.e. sería un gran error hacer una gráfica para mostrar 10 algoritmos de la familia ϵ -greedy.
- ▶ Se pueden modificar los parámetros de las funciones obligatorias de dibujo si se viera conveniente mientras que cumpla con su objetivo (aquí solo se da una propuesta).
- ▶ Tienen libertad para modificar el resto del código del repositorio si se viera necesario.

Implementación de otras familias de algoritmos

Hay varias familias de algoritmo para resolver el problema del bandido de k-brazos. Podemos destacar las siguientes familias.

- ▶ Métodos Greedy: ϵ -greedy, ϵ -decaimiento.
- ▶ Métodos UCB: UCB1, UCB2, UCB1-Tuned, UCB-V, UCB-Decay(a), etc.
- ▶ Métodos de Ascenso del Gradiente: Softmax y Gradiente de Preferencias.

¿Qué debe de hacer?

Debe implementar los métodos ϵ -greedy, UCB1 y Softmax.

Opcionalmente, si quiere, puede implementar ϵ -decaimiento para compararlo con ϵ -greedy y/o UCB2 para compararlo con UCB1.

Realizar un estudio comparativo

¿Qué debe de hacer?

- ▶ Estudie los algoritmos ϵ -greedy, UCB1 y Softmax usando los siguiente tipos de bandidos.
 - Distribución Bernoulli. Cada brazo solo puede dar dos resultados: éxito, 1, o fracaso, 0.
 - Distribución Binomial. A cada brazo se le asocia una distribución $\mathcal{B}(n, p)$.
 - Distribución Normal. Cada brazo sigue una $\mathcal{N}(\mu, \sigma^2)$.

- ▶ **Para cada estudio que realice** construya un notebook diferente.

NOTA

!! Los estudios forman parte de la evaluación. Tenga en cuenta que el diseño de los experimentos forma parte de la evaluación pues conlleva dominio del contenido, justifica el proceso experimental y se tiene capacidad crítica. Evaluar, sintetizar y analizar (formular hipótesis, criticar, experimentar, incorporar, combinar, juzgar, probar, detectar, enlazar resultados, calificar, estimar, comprobar coherencia de lo obtenido, etc ...) son capacitaciones que deben mostrarse en un máster.

- ▶ Utilice siempre la misma semilla para la generación de números aleatorios para que sus gráficas

sean reproducibles.

- Opcionalmente, puede incluir ϵ -decaimiento y/o UCB2.

Realizada la experimentación, desarrolle la componente escrita.

NOTA

!! La documentación es limitada pero el contenido del Github no. *Los notebooks puede contener toda la experimentación que consideréis. P.e. se podría ensayar con varias binomiales o con varias normales, considerando varios valores para sus parámetros. A su vez con las normales se puede considerar una desviación de 1, una desviación de 50 y otra de 100 para las mismas medias. Alternativamente se podría comprobar con 10, 20 o 100 brazos, etc, etc. Los ensayos pueden ser tantos como queráis pero en el documento os tenéis que limitar a los resultados más relevantes y que tengan una justificación desde el punto de vista teórico. P.e. no tendría sentido hacer una experimentación UCB donde cada brazo se ejecuta una vez. Eso es lo que hace un "investigador". Ensayar mucho pero publicar poco: no se puede publicar todo lo que se experimenta y hay que limitarse a una interpretación crítica de los resultados en función de los planteado que, a su vez, viene dado por el conocimiento "profundo" del problema/algoritmos.*

4.2. Grupo 2. Aprendizaje en entornos complejos

El bandido de k -brazos permite estudiar el problema de la explotación y exploración en entornos que no cambian. Sin embargo, lo cierto es que cada toma de decisiones se desarrolla en un contexto. Una forma de abordar el problema es mediante bandidos contextuales. Otra forma, la más habitual, es considerar que las decisiones se rigen por un proceso de decisión de Markov con recompensas (que podrían ser estocásticas). La forma más correcta de resolver este problema es mediante las ecuaciones de Bellman que se pueden resolver mediante cálculo matricial o cálculo iterativo, para lo que se supone que conoce todo (la política correcta, la matriz de transiciones y las recompensas).

Pero lo cierto es que en este tipo de problemas no se suele conocer el modelo que rige el entorno ni se sabe que recompensas nos podemos encontrar y mucho menos cuál es la mejor decisión que se debería de tomar en un estado. Para solucionar el problema se recurre a la experiencia del agente cuando transita por el entorno. Así surge un grupo de técnicas que se basan en aplicar la siguiente secuencia de pasos:

1. El agente observa el estado actual del entorno.
2. Basándose en este estado, el agente toma una acción (de acuerdo a una política) si no está en un estado terminal.
3. El entorno cambia a un nuevo estado como resultado de la acción.
4. El agente recibe una recompensa que se usará para evaluar la acción tomada.
5. Se vuelve al paso 1.

Las técnicas de Monte Carlo esperan a que el agente llegue al estado terminal para considerar el episodio y a partir de él actualiza su política (la toma de decisiones en cada estado) para maximizar la recompensa acumulativa futura. Las técnicas de Diferencias Temporales van actualizando la política conforme va tomando decisiones, sin esperar a llegar a un estado terminal. En medio están las que consideran solo algunas primeras recompensas del episodio. Estas técnicas se basan en usar una representación tabular de las funciones de evaluación de estados y acciones.

En ocasiones la cantidad de estados y acciones es tan enorme, que no queda más remedio que recurrir a funciones aproximadas de las funciones de evaluación, lo que da lugar a todo un conjunto de técnicas basadas en el método del descenso del gradiente (como el uso de redes neuronales). Alternativamente, otro grupo de técnicas se enfocan en optimizar la política directamente lo que da lugar a los métodos Actor-Crítico.

Las tareas a realizar en esta parte son las siguientes.

- Conocer el entorno Gymnasium
- Estudio de algunos algoritmos básicos

A) Conocer el entorno Gymnasium

Gymnasium (<https://gymnasium.farama.org/>) es un entorno que permite ensayar con distintos algoritmos de aprendizaje que se usará para este segundo grupo de algoritmos. Para entender su funcionamiento debe de entender:

- ▶ El último apartado (sección 5.2)
- ▶ La documentación básica de **Gymnasium**.
- ▶ Su aplicación en el notebook

https://github.com/ldaniel-hm/eml_tabular/blob/main/MonteCarloTodasLasVisitas.ipynb.

Notar que **la versión implementada en el notebook no es de Monte Carlo** (pues se calcula *hacia adelante* en vez de *hacia atrás* y casualmente $f(t) = \frac{\sum_{i=1}^t R_i}{t}$ está dando un indicador que parece evolucionar correctamente; pero no deja de ser un algoritmo de aprendizaje sin fundamento teórico).

¿Qué debe de hacer?

Modifique el código para que haga lo siguiente:

- ▶ Construya un agente de acuerdo a las indicaciones de Gymnasium para que aprenda de acuerdo al algoritmo explicado en el notebook. El diseño le servirá para usarlo en el resto de la práctica. La sección 5.2 está basada en la documentación oficial https://gymnasium.farama.org/introduction/train_agent/
- ▶ Añada una gráfica más. Construya una gráfica que utilice $f(t) = \text{len}(\text{episodios}_t)$. ¿Por qué esta gráfica también es un buen indicador de aprendizaje?

B) Estudio de algunos algoritmos básicos

¿Qué debe de hacer?

Desarrolle estudios comparativos entre las distintas técnicas vistas en clase.

Parte 2. Métodos Tabulares:

- ▶ Monte Carlo (on-policy y off-policy)
- ▶ Diferencias temporales (SARSA, Q-Learning)

Parte 3. Control con Aproximaciones:

- ▶ SARSA semi-gradiente
- ▶ Deep Q-Learning.

Construya los correspondientes agentes de aprendizaje y seleccione al menos dos entornos de Gymnasium (<https://gymnasium.farama.org/>) donde uno de ellos **ponga de manifiesto la necesidad** de usar métodos aproximados (p.e. un entorno continuo).

! Importante

Use la representación gráfica o tabla de valores que considere más conveniente para mostrar el aprendizaje de los agentes. Intente usar representaciones o tablas acordes al tipo de aprendizaje.

Notar que no se le pide ningún algoritmo Actor-Crítico.

Pero ¿qué entornos pueden ser utilizados para la experimentación?

- ▶ Los entornos de Gymnasium (https://gymnasium.farama.org/environments/classic_control/). Ya sea Classic Control, Box2D, Toy Text, MuJoCo o Atari.
- ▶ Los entornos externos de Gymnasium como:
 - Bluesky. https://github.com/TUDELFT-CNS-ATM/bluesky-gym/blob/main/bluesky_gym/envs/README.md.
 - Flappy-bird:
 - ★ <https://github.com/robertoschiavone/flappy-bird-env>

- ★ <https://github.com/markub3327/flappy-bird-gymnasium>
 - PyFlyt. <https://taijunjet.com/PyFlyt/index.html>
 - Tetris. <https://github.com/Max-We/Tetris-Gymnasium>
 - Sokoban. <https://github.com/mpSchrader/gym-sokoban>
 - Simplegrid <https://github.com/damat-le/gym-simplegrid>
- **Cualquier otro** que se considere, siempre y cuando su **ejecución** en Colab sea **inmediata**.

! Importante

Los entornos de experimentación no pueden requerir que el docente tenga que estar realizando una instalación manual de software. Todo debe ser reproducible con un simple click en Colab y que responda al esquema general de Gymnasium.

! Cuidado

En Internet existen multitud de notebooks, repositorios y blogs donde se implementan experimentos de RL en entornos como Gymnasium (o versiones similares), utilizando exactamente los mismos algoritmos que se piden. Debéis tener **muchísimo cuidado** con reproducir estructuras experimentales, configuraciones, entornos o resultados que sean **sospechosamente similares** a lo que pueden encontrarse fácilmente en la red.

Esto no se refiere solo a copiar código (que siempre será muy similar). Se refiere a usar también los mismos entornos con los mismos parámetros, con los mismo algoritmos, con los mismos patrones visuales, con réplica de los resultados numéricos, con idénticos diseños experimentales, con las mismas conclusiones, con los mismos estudios futuros, etc ...

La práctica no busca reproducir un experimento estándar de internet. **Buscad vuestro propio estudio experimental, con criterio y justificación.**

Una experimentación demasiado parecida a ejemplos públicos puede considerarse falta grave de originalidad académica, incluso podríamos estar rozando el plagio (aunque todo no sea literalmente copiado).

La originalidad metodológica forma parte de la evaluación. **Tomad este aviso en serio** y recordad que hoy en día los buscadores de OpenAI o Perplexity son capaces de encontrar fácilmente el origen.

P.e. Podéis usar https://gymnasium.farama.org/tutorials/training_agents/ para "inspiraros" pero totalmente prohibido presentar algo "demasiado" similar a los estudios del enlace.

5. Anexos

5.1. Ejemplo reales de uso del k-bandido

Ejemplo de uso una distribución binomial: Optimización de promociones en una app de delivery (tipo Rappi, Uber Eats, DoorDash)

Imagina que trabajas en una app de delivery de comida y tienes $k = 4$ promociones diferentes que puedes ofrecer a los usuarios cada día (cada promoción es un "brazo"): **Promoción A:** 20 % de descuento en pedidos $>30€$; **Promoción B:** Envío gratis + 5€ de crédito; **Promoción C:** 2x1 en postres; **Promoción D:** Descuento fijo de 8€.

Cada día la plataforma decide qué promoción ofrecerle a un lote de $n = 100$ usuarios nuevos que entran por primera vez (o a un segmento específico). No se decide usuario por usuario, sino que se asigna una promoción a todo el lote de 100 usuarios de una vez (esto pasa en la práctica por cuestiones operativas, de presupuesto publicitario o de experimentación por lotes). La recompensa que obtienes al elegir una promoción (brazo) ese día es: número de usuarios (de esos 100) que realmente usaron la promoción (es decir, hicieron un pedido aprovechándola). Esto se modela como una distribución Binomial($n=100, p_i$), donde: $n = 100$ (tamaño fijo del lote de usuarios al que se le muestra la promoción) p_i = probabilidad real (desconocida) de que un usuario use esa promoción si se la ofrecemos

El objetivo del bandido es ir aprendiendo qué promoción tiene el p_i más alto (la que convierte más usuarios en compradores) y asignarle más días (más lotes de 100 usuarios) a medida que pasa el tiempo, maximizando el total de pedidos generados.

Ejemplo de uso de una distribución Bernoulli: Publicidad Online.

Supongamos que un anunciante quiere optimizar su campaña de publicidad en línea eligiendo entre $k = 3$ diferentes anuncios. Cada anuncio tiene una probabilidad desconocida de que un usuario haga clic en él (**CTR - Click-Through Rate**). A_1 tiene un CTR desconocido de p_1 , A_2 tiene un CTR desconocido de p_2 y A_3 tiene un CTR desconocido de p_3 .

El anunciante usa un **modelo de bandido de k -brazos**, donde cada vez que muestra un anuncio (elige un brazo), observa si el usuario hizo clic (éxito = 1) o no (fracaso = 0).

El objetivo del bandido es explotar el anuncio que tiene mayor p_j .

Ejemplo de uso de una distribución Normal: Personalización de recomendaciones.

La personalización de recomendaciones en plataformas de streaming o redes sociales (como YouTube, Spotify, TikTok o Netflix), las recompensas se modelan con distribuciones normales porque el "éxito" no es un simple sí/no (como en Bernoulli) ni un conteo fijo de éxitos (como en Binomial), sino una métrica continua y variable que representa el nivel de interacción del usuario.

Imagina que eres un ingeniero de machine learning en YouTube. La plataforma tiene $k = 5$ algoritmos de recomendación diferentes (cada uno es un "brazo" del bandido). Estos algoritmos podrían ser variantes de un sistema de ranking de videos: **Brazo 1:** Recomendaciones basadas en historial de vistas recientes (enfoque en similitud de contenido). **Brazo 2:** Recomendaciones exploratorias (mezcla de trending topics con algo de aleatoriedad). **Brazo 3:** Recomendaciones optimizadas para retención larga (prioriza videos largos que el usuario ve completos). **Brazo 4:** Recomendaciones basadas en engagement social (videos con muchos likes/comentarios de tus suscripciones).

Cada vez que un usuario abre la app o refresca la página principal, el sistema debe elegir cuál de estos 4 algoritmos usar para generar la lista de recomendaciones para ese usuario (o para un pequeño grupo de usuarios similares en un experimento A/B adaptativo). Esto se hace en tiempo real, miles de veces por segundo. Después de mostrar las recomendaciones, mides **el tiempo total** de visualización (watch time) generado por esa sesión. Por ejemplo: Si el usuario ve 3 videos recomendados durante 15 minutos en total, recompensa = 15 minutos; pero si solo ve 30 segundos y cierra la app, recompensa = 0.5 minutos; etc Incluso algunas recompensas podrían ser negativas.

Aquí tenemos una distribución continua, y no discreta, porque el watch time varía suavemente (de

0 a horas), dependiendo de factores como el interés del usuario, la calidad del video, interrupciones externas, etc. Por lo tanto, se rige por una distribución continua como, p.e., una distribución normal. Experimentalmente se sabe que dependiendo de la distribución considerada algunas técnicas funcionan mejor que otras, destacando algoritmos como UCB o Thompson Sampling.

5.2. Uso básico de Gymnasium

Gymnasium (<https://gymnasium.farama.org/>) es un entorno que permite ensayar con distintos algoritmos de aprendizaje por refuerzo. Se muestra a continuación algunos de trozos de código básicos.

Generar un episodio

```
import gymnasium as gym
env = gym.make('nombreEntorno')

obs, info = env.reset() # Genera un estado inicial

done = False
while not done: # Genera muestras hasta generar el episodio
    action = env.action_space.sample() # Selecciona una acción aleatoria
    next_obs, reward, terminated, truncated, info = env.step(action) # Ejecuta la acción
    done = terminated or truncated # Determina si se terminó.
    obs = next_obs # Pasa al siguiente estado
```

Esquema general de un agente

```
class Agent:
    def __init__(self, env:gym.Env, hiperparámetros):
        """Inicializa todo lo necesario para el aprendizaje"""

    def get_action(self, state) -> Any:
        """
        Indicará qué acción realizar de acuerdo al estado.
        Responde a la política del agente.
        Construir tantas funciones de este tipo como políticas se quieran usar.
        """

    def update(self, obs, action, next_obs, reward, terminated, truncated, info):
        """
        Con la muestra (s, a, s', r) e información complementaria aplicamos el algoritmo.
        update() no es más que el algoritmo de aprendizaje del agente.
        Se añadirá lo necesario para obtener resultados estadísticos, evolución, etc ...
        """
```

Esquema general del aprendizaje episódico de un agente

```
import gymnasium as gym
from agents import AgentMonteCarlo

env = gym.make('nombreEntorno')
agent = AgentMonteCarloOnPolicy(env) # Si queremos usa MC

for episode in tqdm(range(n_episodes)):
    obs, info = env.reset()
    done = False

    # play one episode
    while not done:
        action = agent.get_action(obs)
        next_obs, reward, terminated, truncated, info = env.step(action)

        # update the agent
        agent.update(self, obs, action, next_obs, reward, terminated, truncated, info)

        # update if the environment is done and the current obs
        done = terminated or truncated
    obs = next_obs
```

```
stats = agent.stats() # Retorna los resultados estadísticos, evolución, etc ...  
  
# Mostrar las estadísticas
```

Espacios

Todo entorno tiene un espacio de acciones (acciones válidas) y un espacio de estados (observaciones válidas).

```
print(env.action_space, env.observation_space)
```

Todos los espacios heredan de la superclase `Space`. Hay varios tipos de espacios (<https://gymnasium.farama.org/api/spaces/>). Destacamos:

- ▶ **Discrete**: representa un conjunto de elementos mutuamente excluyentes, numerados de 0 a $n-1$. `env.action_space.n` retorna un entero con el número de elementos del espacio. Por ejemplo, `Discrete(n=4)` puede usarse para un espacio de acción de cuatro direcciones para moverse [izquierda, derecha, arriba o abajo].
- ▶ **Box**: representa un tensor de n dimensiones de números racionales con intervalos $[low, high]$. Por ejemplo, un pedal acelerador con un solo valor entre 0.0 y 1.0 podría codificarse como `Box(low=0.0, high=1.0, shape=(1,), dtype=np.float32)` (el argumento `shape` es una tupla de longitud 1 con un solo valor de 1, lo que nos da un tensor unidimensional con un solo valor). Otro ejemplo de **Box** podría ser una observación de pantalla de Atari que es una imagen RGB de tamaño 210×160 : `Box(low=0, high=255, shape=(210, 160, 3), dtype=np.uint8)`. En este caso, el argumento `shape` es una tupla de tres elementos: la primera dimensión es la altura de la imagen, la segunda es el ancho y la tercera es igual a 3, que corresponden a los tres planos de color para rojo, verde y azul, respectivamente. Entonces, en total, cada observación es un tensor 3D con 100,800 bytes.
- ▶ **Tuple**: nos permite combinar varias instancias de la clase `Space`. Esto nos permite crear espacios de acción y observación de cualquier complejidad que deseemos. Por ejemplo, imagina que queremos crear una especificación de espacio de acción para un automóvil. El automóvil tiene varios controles que podrían cambiarse en cada instante, incluyendo el ángulo del volante, la posición del pedal del freno y la posición del pedal del acelerador. Estos tres controles podrían especificarse mediante tres valores flotantes en una sola instancia de **Box**. Además de estos controles esenciales, el automóvil tiene controles discretos adicionales, como una señal de giro (que podría ser "apagada", "derecha" o "izquierda"), bocina ("encendida" o "apagada"), entre otros. Para combinar todo esto en una sola clase de especificación de espacio de acción, podemos crear `Tuple(spaces=(Box(low=-1.0, high=1.0, shape=(3,), dtype=np.float32), Discrete(n=3), Discrete(n=2)))`.

Hay más tipos de espacios. Consulte <https://gymnasium.farama.org/api/spaces/#fundamental-spaces>

Wrapper

En ocasiones se quiere extender la funcionalidad del entorno de alguna manera genérica. Por ejemplo, un entorno te proporciona algunas observaciones, pero deseas acumularlas en algún buffer y proporcionar al agente las últimas N observaciones.

Hay muchas situaciones de este tipo: te gustaría "envolver" el entorno existente y añadir alguna lógica extra que haga algo más. Gymnasium proporciona un marco general para estas situaciones con la clase `Wrapper`.

La clase `Wrapper` hereda de la clase `Env`. Su constructor acepta un solo argumento: la instancia de la clase `Env` que se va a "envolver". Para añadir funcionalidad extra, necesitas redefinir los métodos que deseas extender, como `step()` o `reset()`. **El único requisito es llamar al método original de la superclase.**

Gymnasium proporciona muchas subclases de la clase `Wrapper` (<https://gymnasium.farama.org/api/wrappers/table/>). Comentamos algunas que permiten filtrar solo una parte específica de la información.

- **ObservationWrapper**: Necesitas redefinir el método `observation(obs)` de la clase padre. El argumento `obs` es una observación del entorno envuelto, y este método debe devolver la observación que se le dará al agente.
- **RewardWrapper**: Esta supone modificar el método `reward(rew)`, que podría modificar el valor de la recompensa dada al agente.
- **ActionWrapper**: Necesitas sobrescribir el método `action(act)`, que podría ajustar la acción pasada al entorno envuelto para el agente.

Para hacerlo un poco más práctico, imaginemos una situación en la que quiere aplicar una política ϵ -greedy, entonces una forma de hacerlo es:

```
class RandomActionWrapper(ActionWrapper):
    def __init__(self, env, epsilon=0.1):
        super().__init__(env)
        self.epsilon = epsilon

    def action(self, action):
        if random.random() < self.epsilon:
            return self.env.action_space.sample()
        return action

if __name__ == "__main__":
    env = RandomActionWrapper(gym.make("CartPole-v0"))
```

Notar que en este ejemplo se delega la política del agente en el entorno. Es decir, estamos modificando el comportamiento del entorno con este Wrapper. Se aconseja que las políticas se incluyan mejor en el agente.

Hay un Wrapper que permite grabar vídeos de los episodios que se generen en el entorno usando el método `render()`. No se recomienda su uso **salvo que quiera mostrar de forma animada la política óptima final obtenida después del entrenamiento**. En https://github.com/ldaniel-hm/eml_tabular/blob/main/EjemploGeneracionVideos.ipynb tiene un ejemplo de como usarlo desde Colab.

5.3. Usar PyTorch para Definir una QNet

Una forma para definir un Red con PyTorch es la siguiente

```
import torch
import torch.nn as nn

# Definición de la Red Neuronal Profunda DQN
class DQN_Network(nn.Module):
    """
    Red neuronal para el algoritmo Deep Q-Network (DQN).
    Se compone de capas totalmente conectadas (FC) con activaciones ReLU.
    """

    def __init__(self, num_actions, input_dim):
        """
        Inicializa la red neuronal.

        Parámetros:
            num_actions (int): Número total de acciones posibles en el entorno.
            input_dim (int): Dimensión del espacio de estados de entrada.
        """

        super(DQN_Network, self).__init__()

        # Definición de las capas de la red neuronal
        self.FC = nn.Sequential(
            nn.Linear(input_dim, 12), # Capa de entrada con 12 neuronas
            nn.ReLU(inplace=True), # Función de activación ReLU
            nn.Linear(12, 8), # Capa oculta con 8 neuronas
            nn.ReLU(inplace=True), # Otra activación ReLU
            nn.Linear(8, num_actions) # Capa de salida con 'num_actions' neuronas
        )
```

```
# Inicialización de pesos usando He initialization (Kaiming Uniform)
for layer in [self.FC]:
    for module in layer:
        if isinstance(module, nn.Linear):
            nn.init.kaiming_uniform_(module.weight, nonlinearity='relu')

def forward(self, x):
    """
    Propagación hacia adelante de la red para calcular los valores Q.

    Parámetros:
        x (torch.Tensor): Estado de entrada representado como tensor.

    Retorna:
        Q (torch.Tensor): Valores Q para cada acción posible.
    """

    Q = self.FC(x) # Pasa el estado a través de la red neuronal
    return Q
```

5.4. Una Recomendación para Semillas Fijas

Una forma que se recomienda para iniciar la tarea siempre con la misma semilla es la siguiente (Ojo, puede que no te funcione del todo):

```
import os
import gc
import torch
import numpy as np
import gymnasium as gym

# Configuración del dispositivo (CPU o GPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")

# Liberación de memoria para evitar problemas de consumo en GPU
gc.collect() # Ejecuta el recolector de basura de Python
torch.cuda.empty_cache() # Vacía la caché de memoria en GPU

# Depuración de errores en CUDA
os.environ['CUDA_LAUNCH_BLOCKING'] = '1' # Muestra errores de CUDA en el punto exacto donde ocurren

# Configuración de la semilla para reproducibilidad
seed = 2024 # Se define una semilla fija

# Fijar la semilla en NumPy
np.random.seed(seed) # Para generar números aleatorios consistentes en NumPy
np.random.default_rng(seed) # Establece una instancia del generador de NumPy con la misma semilla

# Fijar la semilla en Python
os.environ['PYTHONHASHSEED'] = str(seed) # Evita variabilidad en hashing de Python

# Fijar la semilla en PyTorch
torch.manual_seed(seed) # Asegura resultados reproducibles en operaciones de PyTorch

if torch.cuda.is_available(): # Si hay GPU disponible
    torch.cuda.manual_seed(seed) # Fija la semilla para la GPU
    torch.backends.cudnn.deterministic = True # Hace las operaciones de CUDNN determinísticas
    torch.backends.cudnn.benchmark = False # Desactiva optimizaciones de CUDNN para evitar variabilidad

# Fijar la semilla en Gymnasium
def make_env(env_name):
    env = gym.make(env_name)
    env.reset(seed=seed) # Establece la semilla en el entorno de Gymnasium
    return env

# Ejemplo de creación de un entorno con semilla
env_name = "CartPole-v1" # Cambiar según el entorno deseado
env = make_env(env_name)
```